

Análisis de Series de Tiempo II

Forecasting Multivariado y Global Forecasting Models

MSc. Fernando Silva



- Clase 4
- Clase 5
- Clase 6
- Clase 7 (martes)

Clase 8 (21 de agosto viernes)

Agenda

- Repaso
- Forecasting multivariado
- Global forecasting models
- Embeddings categóricos
- Forecasting Jerárquico

Repaso

Repaso

Estrategia (Clase 1)	Arquitectura (hoy)	Fortaleza	Costo
Recursive	Rollout autoregresivo (DeepAR-style)	H flexible	Acumula error, exposure bias
Direct	H cabezas sobre h_T	Sin acumulación	Sin coherencia entre horizontes
MIMO	Encoder-Decoder / Seq2Seq	Coherencia conjunta	Requiere teacher forcing

Las estrategias son ortogonales a la arquitectura:
En la clase 3 solo les pusimos una red adentro.

Introducción

Series de Tiempo Multivariadas

Las series de tiempo multivariadas están en todas partes:

Magnitudes correlacionadas registradas simultáneamente

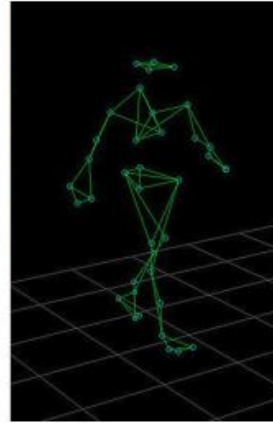
Ejemplo: número de hospitalizaciones, número de pacientes en unidades de cuidados intensivos (UCI) y número de fallecimientos durante la pandemia de COVID-19.

Trayectorias en 2D y 3D

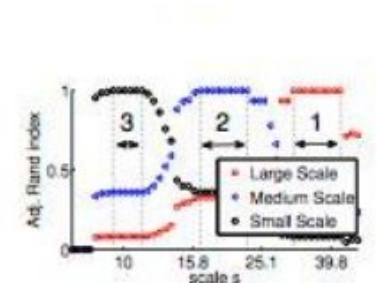
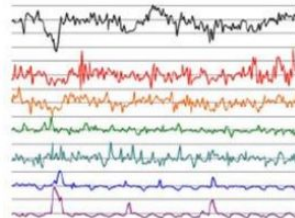
Ejemplo: captura de movimiento (motion capture), seguimiento del cursor del mouse (mouse tracking) y seguimiento ocular (eye tracking).

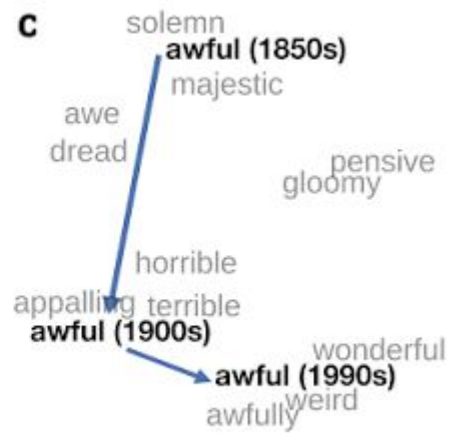
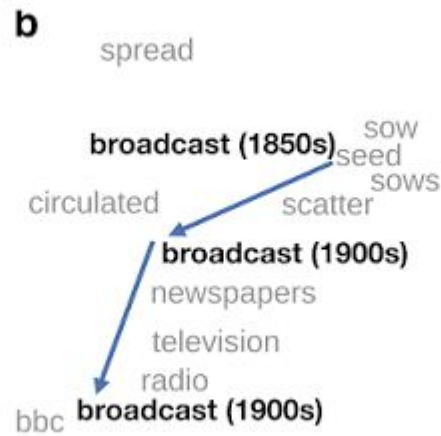
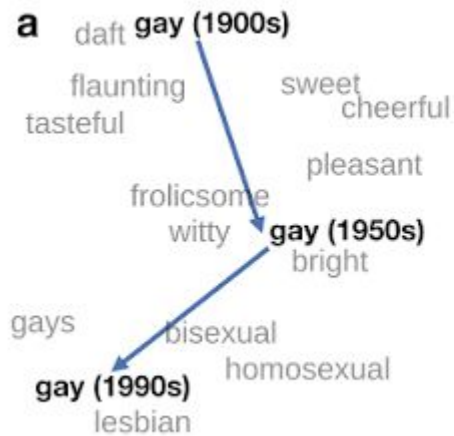
Redes de sensores

Ejemplo: monitoreo ambiental, redes inalámbricas, redes sociales y otros sistemas distribuidos.



Sensor Data





Series de Tiempo Multivariadas

¿Cómo lo procesamos?

Dos soluciones:

Procesar cada dimensión de forma independiente (como vimos en las clases anteriores)

Utilizar modelos o enfoques específicos que tengan en cuenta las relaciones o dependencias entre las distintas dimensiones.

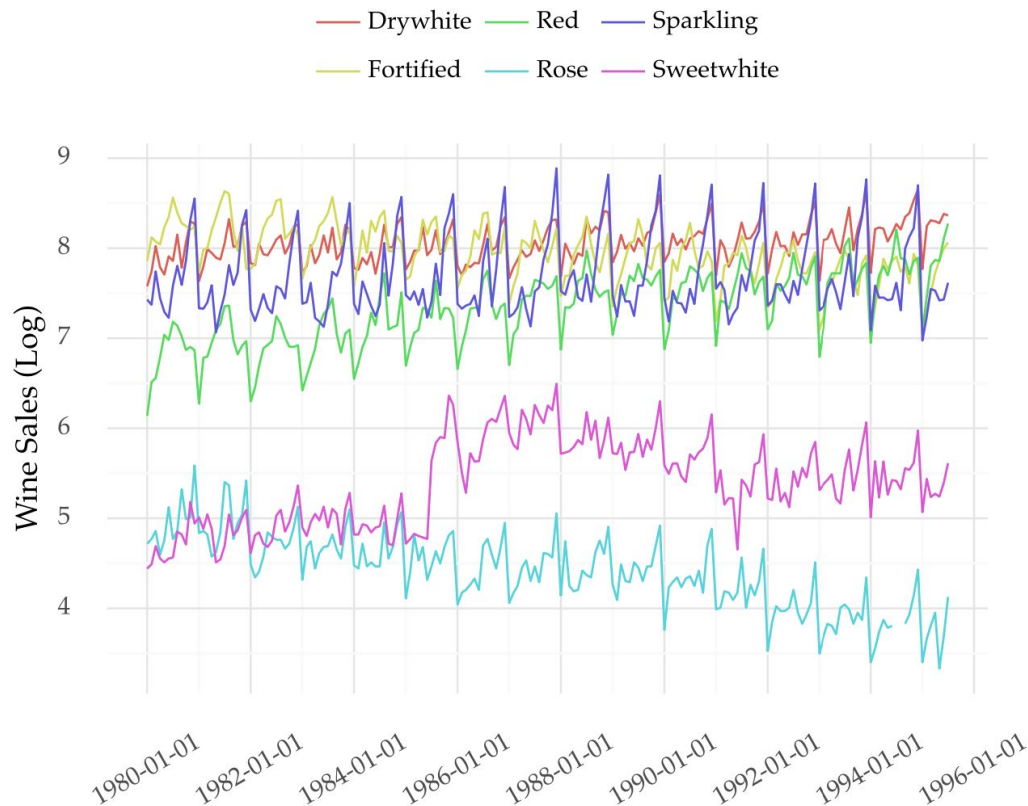
Notación:

- Una serie de tiempo de longitud N con D variables (dimensiones).
- Cada observación temporal $\mathbf{x}[n]$ es un vector de longitud D , que contiene los valores de todas las variables en el instante n .
- Toda la serie de tiempo puede almacenarse en una matriz $\mathbf{X}$ de tamaño $N \times D$, donde:
 - N : número de instantes de tiempo (filas).
 - D : número de variables o dimensiones (columnas).

$$\mathbf{X} = \begin{bmatrix} x_1[1] & \cdots & x_D[1] \\ \vdots & \cdots & \vdots \\ x_1[N] & \cdots & x_D[N] \end{bmatrix}$$

Series de Tiempo Multivariadas

Ejemplo



Exploración de Series Multivariadas

[LINK](#)



Forecasting Multivariado

Formalización: el conjunto de información F_t

- Todo problema de forecasting se define por tres cosas:

Target

¿qué variable
queremos predecir?

Horizonte

¿cuántos pasos hacia
el futuro queremos
predecir?

Conjunto de información F_t disponible en t

¿qué información conocemos cuando hacemos la
predicción?

- Tenemos:

Univariado

Solo se manejaba el
historial de la variable
objetivo y .

AR, ARIMA, ETS

Multivariado

Pasado del target, pasado de
covariables, y covariables
conocidas hasta el horizonte.

DeepAR + covariables, TFT

Multi-step $\neq$ Multivariado $\neq$ Multi-output

Multi-step (Clase 1)

¿Cuántos pasos hacia el futuro queremos predecir?

Estrategias recursiva, directa, **MIMO**

Multivariado (input)

¿Cuántas variables observa el modelo?

Dimensión del input por paso, el modelo observa **d** variables en cada instante

Multi-output (target)

¿Cuántas variables queremos predecir?

Predecir varias series a la vez, $\hat{y}_{t+1} \in \mathbb{R}^k$; **VAR** es el caso clásico lineal

- Veremos multivariado en input con target escalar:

$$X \in \mathbb{R}^{L \times d} \quad a \quad \hat{y} \in \mathbb{R}$$

¿Qué problema resuelve el multivariado?

Forecasting Univariado

Supone que toda la información predictiva sobre y_{t+H} está contenida en la propia trayectoria pasada de y , supuesto fuerte y casi siempre falso

Estimamos:

$$P(y_{t+H} | y_{\leq t})$$



Forecasting Multivariado

Es una decisión sobre el conjunto de información, el mismo LSTM, el mismo Transformer y el mismo LightGBM soportan ambos regímenes

Estimamos:

$$P(y_{t+H} | y_{\leq t}, x_{\leq t}, z_{\leq t+H})$$

Tres fuentes de información que el univariado descarta:

- **Covariables exógenas** (variables externas a la serie objetivo que afectan su comportamiento, pero que no son generadas por la propia serie: clima, precios, etc)
- **Muchas series de tiempo no existen de manera aislada**, sino que forman parte de un sistema donde unas variables afectan a otras.

el objeto que cambia es el condicionamiento

Una covariable justifica su inclusión sólo si reduce el error fuera de muestra, no si mejora el ajuste ni si la correlación es alta

¿Qué cambia de verdad en la arquitectura?

Univariado

$$\hat{y}_{t+H} = f(y_{t-L+1}, \dots, y_t), input \in \mathbb{R}^{L \times 1}$$

Multivariado

$$\hat{y}_{t+H} = f(X_{t-L+1}, \dots, X_t), input \in \mathbb{R}^{L \times d}$$
$$con X_t = [y_t, x_t^{(1)}, \dots, x_t^{(d-1)}]$$

- En el LSTM:

$$W_{ix} \text{ pasa de } \mathbb{R}^{h \times 1} \text{ a } \mathbb{R}^{h \times d}$$

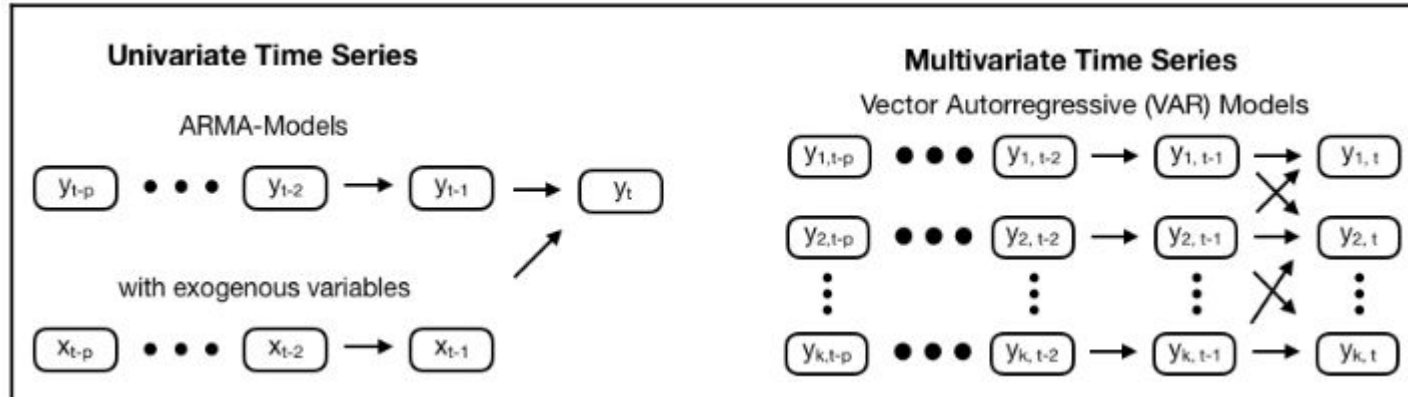
las cuatro compuertas ahora mezclan canales en cada paso (channel mixing)

- Parámetros extra
- El estado oculto h_t se vuelve un resumen comprimido de la historia conjunta de todas las variables, no hay un estado por canal
- Consecuencia de diseño: si una covariable es ruido, contamina el estado compartido; la selección de variables es una decisión de arquitectura, no un detalle

De las VAR a las NN

Vector Autoregression (VAR)

- Un modelo AR aprende del historial de una sola serie para predecir su siguiente valor.
- En cambio, VAR aprende del historial de todas las series y usa esa información conjunta para realizar las predicciones.

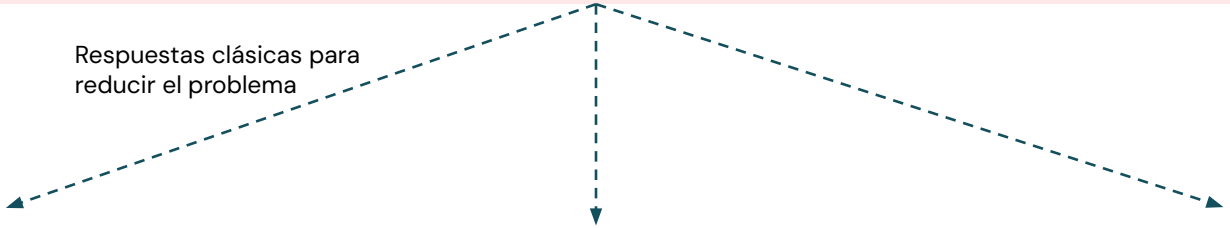


De las VAR a las NN

Vector Autoregression (VAR)

Maldición de la dimensionalidad: A medida que aumentan el número de series, el modelo VAR debe estimar miles de parámetros, por lo que necesita muchos más datos para aprender correctamente.

Respuestas clásicas para
reducir el problema



```
graph TD; A[Respuestas clásicas para reducir el problema] -.-> B[VAR Bayesiano]; A -.-> C[Reducción de rango]; A -.-> D[Sparse VAR];
```

VAR Bayesiano

Añade conocimiento previo que hace que muchos coeficientes se acerquen a cero, reduciendo el sobreajuste.

Reducción de rango

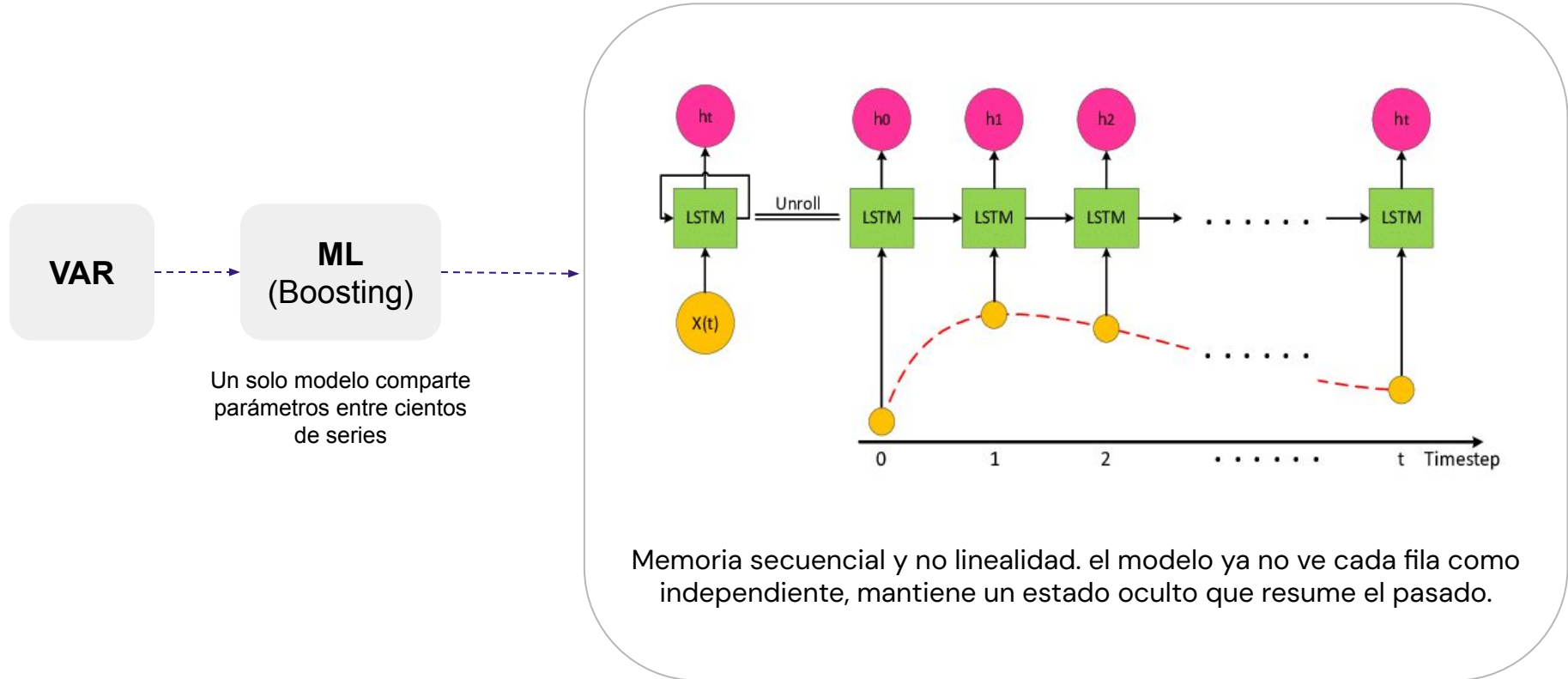
Asume que las relaciones entre las variables pueden describirse con menos parámetros de los que usa un VAR completo.

Sparse VAR

Utiliza penalización L1 para forzar que muchos coeficientes sean exactamente cero, conservando solo las relaciones más importantes.

De las VAR a las NN

RNN / LSTM / GRU



De las VAR a las NN

DeepAR

- Es un modelo generativo autoregresivo: en cada paso de tiempo, en vez de predecir un valor puntual, **predice los parámetros de una distribución de probabilidad** para el siguiente valor.
- La serie completa se modela como una **secuencia de estas distribuciones** condicionales, cada una condicionada a todo lo observado hasta ese momento.
- **Es global:** una sola red se entrena sobre muchas series distintas, **no una red por serie**, los parámetros de la red son compartidos, eso es lo que le permite generalizar entre series y funcionar razonablemente bien incluso con series que tienen poca historia.

Clasificación de covariables

Estáticas

Invariantes en el tiempo

(industria, región, categoría),
condicionan el modelo entero,
no entran al eje temporal

Dinámicas conocidas a futuro

Sí cambian con el tiempo. Pero ya
conocemos sus valores futuros antes
de predecir.

Calendario, feriados, promociones
planificadas, etc

Dinámicas desconocidas a futuro

Cambian con el tiempo. Pero no
sabemos su valor futuro.

Temperatura observada, tráfico,
precios de terceros, etc

- **Consecuencia arquitectónica en un encoder–decoder:** x_t alimenta solo el encoder, z_t alimenta encoder y decoder; s condiciona a ambos vía context vectors
- **Ejemplo: DeepAR** recibe z_t futuro en cada paso del decoder; esa es la razón estructural por la que funciona en retail con feriados

Causalidad de Granger

- Evalúa si el historial de una variable contiene información útil para predecir otra.
- Se dice que **X** Granger-cause a **Y** si incorporar los valores pasados de **X** mejora significativamente la predicción de **Y**, en comparación con utilizar únicamente el pasado de **Y**.
- La prueba se basa en comparar dos modelos:
 - **Modelo 1:** utiliza solo el pasado de **Y**.
 - **Modelo 2:** utiliza el pasado de **Y** y el pasado de **X**.
- Si el segundo modelo ofrece una mejora estadísticamente significativa, se concluye que existe causalidad de Granger.
- **La causalidad de Granger indica capacidad predictiva, no una relación de causa y efecto en sentido estricto.**

Arquitectura del multivariado

Channel Mixing vs Channel Independence

En lugar de una sola variable ($L \times 1$), el modelo recibe una matriz ($L \times d$), donde L es la longitud de la secuencia y d el número de variables (canales) observadas en cada instante.

Channel Mixing

Modelos como las LSTM o GRU mezclan toda la información de los d canales en cada paso temporal para construir un único estado oculto que resume la historia de todas las variables.

Si una covariable es ruidosa o irrelevante, su información también entra en ese estado compartido y puede perjudicar la predicción.

Channel Independence

Procesa cada variable por separado (generalmente compartiendo los mismos pesos) y combina la información solo al final, reduciendo la interferencia entre canales.

Aunque es menos expresivo, utiliza menos parámetros y reduce el sobreajuste, especialmente cuando hay muchas variables y pocos datos.

Mezclar variables supone que existen relaciones importantes entre ellas; si los datos no respaldan esa hipótesis, un modelo más simple e independiente suele generalizar mejor.

Arquitectura del multivariado

Channel Mixing vs Channel Independence

Published as a conference paper at ICLR 2023

A TIME SERIES IS WORTH 64 WORDS: LONG-TERM FORECASTING WITH TRANSFORMERS

Yuqi Nie¹*; Nam H. Nguyen²*; Phanwadee Sinthong², Jayant Kalagnanam²

¹Princeton University ²IBM Research
ynie@princeton.edu, nnguyen@us.ibm.com, Gift.Sinthong@ibm.com,
jayant@us.ibm.com

ABSTRACT

We propose an efficient design of Transformer-based models for multivariate time series forecasting and self-supervised representation learning. It is based on two key components: (i) segmentation of time series into subseries-level patches which are served as input tokens to Transformer; (ii) channel-independence where each channel contains a single univariate time series that shares the same embedding and Transformer weights across all the series. Patching design naturally has three-fold benefit: local semantic information is retained in the embedding; computation and memory usage of the attention maps are quadratically reduced given the same look-back window; and the model can attend longer history. Our channel-independent patch time series Transformer (PatchTST) can improve the long-term forecasting accuracy significantly when compared with that of SOTA Transformer-based models. We also apply our model to self-supervised pre-training tasks and attain excellent fine-tuning performance, which outperforms supervised training on large datasets. Transferring of masked pre-trained representation on one dataset to others also produces SOTA forecasting accuracy.

A.7.1 CHANNEL-INDEPENDENCE VS CHANNEL-MIXING

We find 3 key factors that makes channel-independent models more preferable:

- Adaptability: Since each time series is passed through the Transformer separately, it generates its own attention maps. That means different series can learn different attention patterns for their prediction, as shown in Figure 6. In contrast, with the channel mixing approach, all the series share the same attention patterns, which may be harmful if the underlying multivariate time series carries series of different behaviors. Figure 6 reveals an interesting observation that the prediction of unrelated time series relies on different attention patterns while similar series can produce similar maps (e.g. series 11, 25, and 81 contain similar patterns while they are different from others). We suspect this adaptability is one of the main reasons why PatchTST performs much better forecasting than Informer and other channel-mixing models.
- Channel-mixing models may need more training data to match the performance of the channel-independent ones. The flexibility of learning cross-channel correlations could be a double-edged sword, because it may need much more data to learn the information from different channels and different time steps jointly and appropriately, while channel-independent models only focus on learning information along the time axis. We examine this assumption by experiments where we train the models with varying training data size and the result is shown on left panel of Figure 7. It is clear that channel-independent models converges faster against the size of training data. As what we have observed in the figure and Table 2, the size of those widely used time series datasets may not be large enough for channel-mixing models to obtain similar performances in supervised learning.
- Channel-independent models are less likely to overfit data during training. We record the MSE loss on test data and plot on the right panel of Figure 7. Channel-mixing models show overfitting after a few initial epochs, while channel-independent models continue optimizing the loss with more training epochs. The best trained models are determined by validation data, which are approximately the lowest points in the test loss curves. It is clear that the forecasting performance of channel-independent models are better.

Propone **channel-independence**, donde cada canal contiene una serie univariada que comparte el mismo embedding y los mismos pesos del Transformer entre todas las series.

Fuente: Nie, Y., Nguyen, N. H., Sinthong, P., & Kalagnanam, J. (2023). A time series is worth 64 words: Long-term forecasting with transformers.

Arquitectura del multivariado

Channel Mixing vs Channel Independence

The Capacity and Robustness Trade-off: Revisiting the Channel Independent Strategy for Multivariate Time Series Forecasting

Lu Han, Han-Jia Ye, De-Chuan Zhan
State Key Laboratory for Novel Software Technology, Nanjing University
{hanlu, yehj}@lamda.nju.edu.cn, zhandc@nju.edu.cn

ABSTRACT

Multivariate time series data comprises various channels of variables. The multivariate forecasting models need to capture the relationship between the channels to accurately predict future values. However, recently, there has been an emergence of methods that employ the Channel Independent (CI) strategy. These methods view multivariate time series data as separate univariate time series and disregard the correlation between channels. Surprisingly, our empirical results have shown that models trained with the CI strategy outperform those trained with the Channel Dependent (CD) strategy, usually by a significant margin. Nevertheless, the reasons behind this phenomenon have not yet been thoroughly explored in the literature. This paper provides comprehensive empirical and theoretical analyses of the characteristics of multivariate time series datasets and the CI/CD strategy. Our results conclude that the CD approach has higher capacity but often lacks robustness to accurately predict distributionally drifted time series. In contrast, the CI approach trades capacity for robust prediction. Practical measures inspired by these analyses are proposed to address the capacity and robustness dilemma, including a modified CD method called Predict Residuals with Regularization (PRReg) that can surpass the CI strategy. We hope our findings can raise awareness among researchers about the characteristics of multivariate time series and inspire the construction of better forecasting models.

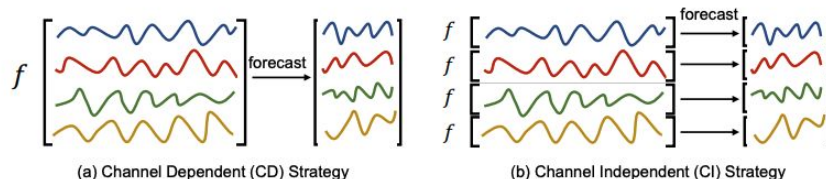


Figure 1: Comparison of two training strategies for Multivariate Time Series Forecasting (MTSF) tasks. The left shows the Channel Dependent (CD) strategy where all the channels are taken as input and forecasted future values depend on the history of all the channels. The right shows the Channel Independent (CI) strategy, which treats the multivariate series as multiple univariate series and trains a unified model on these series. The prediction of each channel depends solely on its own historical values, and the relationship between different channels is ignored.

Mientras **channel dependence** se beneficia idealmente de mayor capacidad, **channel independence** mejora mucho el rendimiento por escasez de muestras. “El mixing agrega parámetros que sobreajustan dependencias cruzadas espurias cuando d es grande y n pequeño”

Fuente: Han, L., Ye, H.-J., & Zhan, D.-C. (2023). The capacity and robustness trade-off: Revisiting the channel independent strategy for multivariate time series forecasting

Arquitectura del multivariado

Escalado y normalización con d variables

Escalas diferentes: si una variable tiene valores mucho mayores que las demás (por ejemplo, ventas en miles y temperatura en decenas), dominará el entrenamiento y el modelo prestará menos atención a las otras.

Estandarización por variable

Cada variable debe normalizarse usando su propia **media (μ)** y **desviación estándar (σ)** calculadas solo con los datos de entrenamiento, para evitar data leakage.

Normalizar el objetivo (target)

La variable que se quiere predecir también suele estandarizarse para estabilizar el entrenamiento; al finalizar, las predicciones se convierten nuevamente a la escala original.

Variables Categóricas

No se estandarizan; si tienen pocas categorías se representan con one-hot encoding, y si tienen muchas, mediante embeddings aprendidos por el modelo.

Datos no estacionarios

Si la distribución cambia con el tiempo, pueden aplicarse técnicas como diferenciación, transformación logarítmica o RevIN, que normaliza cada muestra de entrada y luego revierte esa normalización en la salida.



RevIN se adapta a cambios en la distribución de los datos (distribution shift) sin depender de estadísticas globales, lo que mejora la robustez del modelo.

Global Forecasting Models

¿Por qué existe el enfoque global?

Enfoque local:

Los modelos clásicos, como ARIMA o ETS, entrenan un modelo independiente para cada serie temporal, ya que cada una se considera un problema diferente.

Enfoque global:

En lugar de entrenar miles de modelos, se entrena un único modelo con todas las series, compartiendo los mismos parámetros y aprendiendo patrones comunes entre ellas.

- El enfoque global surge por un problema de escala: empresas con miles o millones de series temporales no pueden mantener un modelo distinto para cada una.
- Las redes neuronales tienen millones de parámetros y necesitan grandes cantidades de datos. Una sola serie suele ser insuficiente, pero al entrenar con muchas series el modelo dispone de información suficiente para aprender.
- En series temporales, los datos suelen crecer más en número de series que en longitud de cada serie. Por eso, el enfoque global no solo reduce costos de entrenamiento, sino que hace posible aplicar deep learning de forma efectiva.

Enfoque Global

- Un enfoque global aprende un único conjunto de parámetros utilizando todas las series temporales al mismo tiempo, compartiendo la misma función de predicción entre ellas.
- No significa que prediga una serie promedio: cada serie genera predicciones diferentes según sus propios datos de entrada.
- Un enfoque global no es lo mismo que un modelo multivariado, el primero entrena con muchas series univariadas, mientras que el segundo utiliza varias variables para realizar una predicción.
- La decisión entre local y global depende más de la complejidad del modelo y la cantidad de datos disponibles que de si las series son idénticas o diferentes.

Ingeniería del pooling

Escalado por serie:

Sin normalizar, la pérdida del pool queda dominada por las series de mayor magnitud absoluta y el modelo ignora efectivamente a las pequeñas.

Split correcto:

Temporal dentro de cada serie, y pooling después, solo con los tramos de entrenamiento, agrupar primero y cortar por índice deja series enteras fuera del entrenamiento

Z-score por serie:

μ y σ estimados exclusivamente sobre el tramo de entrenamiento de cada serie, usar estadísticas de la serie completa filtra información del futuro (leakage)

Muestreo del pool:

Apilar sin pesos hace que cada ventana pese igual, así que las series largas dominan; ponderar para que cada serie pese igual es una alternativa legítima, no hay opción neutra, el muestreo es un hiperparámetro

Global Forecasting Models

[LINK](#)



Global Forecasting Models

- **En la práctica rara vez tenemos una sola serie: tenemos cientos o miles.** Ante ese escenario hay dos estrategias:
 - **local**, un modelo independiente por serie.
 - **global**, un único modelo entrenado con todas a la vez

Y de forma transversal, cada serie puede predecirse solo con su propio pasado (**univariado**) o incorporando covariables (**multivariado**).

- **Channel independence vs. channel mixing describe si el modelo mezcla información entre variables/series dentro del forward pass.**
- **Cuándo no usar global**
 - Pocas series (<10), dinámicas muy dispares, o necesidad de interpretabilidad estadística por serie.

Embeddings Categóricos

El problema de la identidad, formalizado

- El modelo global del bloque anterior estima $E[y_{t+H}|ventana]$, marginalizando sobre las series:
 - **ventanas idénticas producen predicciones idénticas, sin importar de qué serie provengan.**
- Lo que en realidad se quiere estimar es $E[y_{t+H}|ventana, serie = i]$, condicionar en la identidad sin renunciar a los pesos compartidos que dan la ganancia del pooling

Solución descartada, Un modelo por serie

Reintroduce exactamente el problema local que el pooling vino a resolver

El problema de la identidad, formalizado

- El modelo global del bloque anterior estima $E[y_{t+H}|ventana]$, marginalizando sobre las series:
 - **ventanas idénticas producen predicciones idénticas, sin importar de qué serie provengan.**
- Lo que en realidad se quiere estimar es $E[y_{t+H}|ventana, serie = i]$, condicionar en la identidad sin renunciar a los pesos compartidos que dan la ganancia del pooling

Solución descartada, Un modelo por serie

Reintroduce exactamente el problema local que el pooling vino a resolver

Solución descartada , Una variable dummy por serie

Escala mal en cardinalidad alta y no generaliza a series nuevas

El problema de la identidad, formalizado

- El modelo global del bloque anterior estima $E[y_{t+H}|ventana]$, marginalizando sobre las series:
 - **ventanas idénticas producen predicciones idénticas, sin importar de qué serie provengan.**
- Lo que en realidad se quiere estimar es $E[y_{t+H}|ventana, serie = i]$, condicionar en la identidad sin renunciar a los pesos compartidos que dan la ganancia del pooling

Solución descartada, Un modelo por serie

Reintroduce exactamente el problema local que el pooling vino a resolver

Solución descartada , Una variable dummy por serie

Escala mal en cardinalidad alta y no generaliza a series nuevas

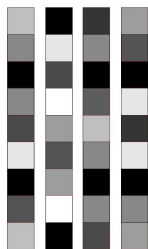
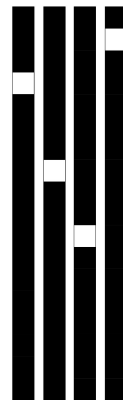
Solución adoptada, La identidad entra como input aprendible

Un vector $e_i \in \mathbb{R}^k$ por serie, entrenado por backpropagation junto al resto de los parámetros

- El resultado es un solo modelo, N comportamientos, una interpolación continua entre el extremo global (todos iguales) y el extremo local (todos distintos), a costo de $k \cdot N$ parámetros adicionales

One-hot vs Embedding

- **Dimensionalidad:** one-hot crece linealmente con el número de series
- **Esparsidad estadística:** cada columna del one-hot se entrena únicamente con las observaciones de esa categoría, las categorías raras reciben estimaciones ruidosas y las nuevas no reciben nada
- **Geometría:** el one-hot hace a todas las categorías ortogonales y equidistantes, no existe forma de expresar que dos series son parecidas, aunque lo sean



- **Embedding:** la red decide la geometría; series con dinámicas similares convergen a vectores cercanos porque reciben gradientes de error similares durante el entrenamiento
- **Implementación:** una capa de embedding es una lookup table diferenciable

El costo de la identidad

- La "identidad" es el vector que le dice al modelo de qué serie viene cada ventana. El modelo global del bloque anterior es amnésico: recibe 12 meses de números y no sabe si son de minería o de comercio. El embedding es ese dato faltante.
- Puede entrar en tres puntos de la red:
 - Concatenada a la serie en cada paso temporal
 - En el estado oculto inicial
 - Pegada al último estado antes de la salida

Cuanto más adentro entra, más capacidad de diferenciar y menos queda de la regularización implícita del pooling

- La identidad le devuelve al modelo la capacidad de memorizar por serie que el pooling le había quitado

Embeddings Categóricos

[LINK](#)



Forecasting jerárquico

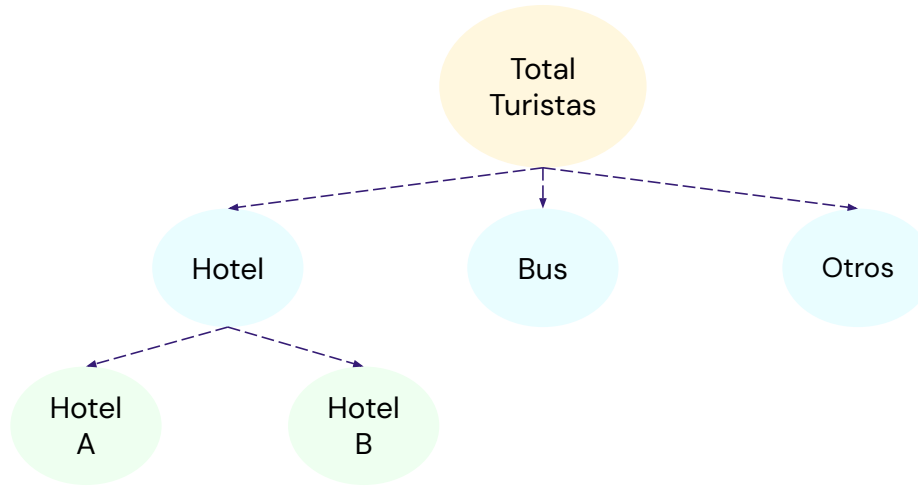
Introducción

- El forecasting jerárquico es un enfoque de predicción de series de tiempo donde los datos tienen una estructura organizada en diferentes niveles de agregación, y el objetivo es generar pronósticos que sean coherentes entre todos los niveles de la jerarquía.
- No se predice una sola serie temporal aislada, sino un conjunto de series relacionadas mediante una estructura jerárquica.



Forecasting Jerárquico

- En muchos problemas reales, los datos poseen naturalmente una organización por niveles.
- Por ejemplo, una empresa turística puede tener la cantidad de visitantes organizada así:



- Existe una relación matemática: $\text{Total} = \text{Hotel} + \text{Bus} + \text{Otros}$ y $\text{Hotel} = \text{HotelA} + \text{HotelB}$
- Entonces todas las series están relacionadas

Forecasting Jerárquico

- Si hacemos forecasting tradicional de manera independiente:
 - Pronosticamos Total
 - Pronosticamos Hotel
 - Pronosticamos Bus
 - Pronosticamos Otros

puede ocurrir algo inconsistente:

Forecast Total = 1000 turistas

Forecast Hotel = 700

Forecast Bus = 400

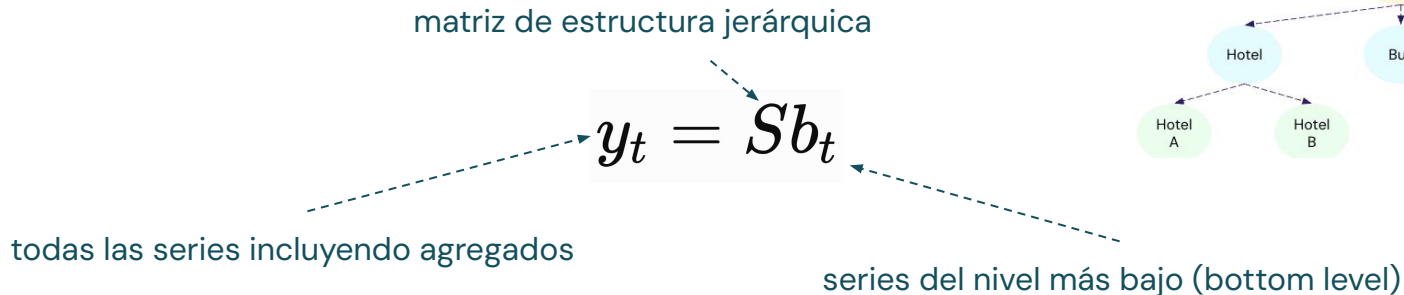
Forecast Otros = 200

Suma de componentes = 1300

- El forecasting jerárquico busca evitar este problema.

Definición Formal

- Sea un conjunto de series temporales organizadas en una jerarquía. Denotamos: y_t como el vector de todas las series observadas en el tiempo t .
- Estas series están relacionadas mediante una matriz de agregación:



- Por ejemplo:

Nivel inferior:

$$b_t = \begin{bmatrix} Hotel_A \\ Hotel_B \\ Bus \\ Otros \end{bmatrix}$$

La **matriz S** construye los niveles superiores:

$$S = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Entonces:

$$y_t = \begin{bmatrix} Total \\ Hotel \\ Bus \\ Otros \end{bmatrix}$$

Objetivo Principal

- El objetivo es obtener pronósticos:

$$\hat{y}_{t+h}$$

que cumplan:

$$\hat{y}_{t+h} = S\hat{b}_{t+h}$$

Los pronósticos deben respetar las relaciones matemáticas de la jerarquía.

- Ejemplo de pronóstico coherente:

Total = 1000

Hotel = 600

Bus = 250

Otros = 150

Suma de componentes = 1000

Métodos Principales de Forecasting

Bottom-Up

Idea:

- Se pronostican únicamente las series inferiores.
- Luego se agregan los resultados.

Primero

Hotel A: 300
Hotel B: 200
Bus: 400
Otros: 100

Luego

$$\text{Total} = 300 + 200 + 400 + 100$$

Ventajas

- Conserva toda la información específica de las series base.
- Captura mejor las dinámicas locales de cada serie.
- Garantiza coherencia al obtener los niveles superiores mediante agregación.

Desventajas

- Requiere pronosticar un gran número de series, aumentando el costo computacional.
- Las series del nivel inferior suelen ser más ruidosas y volátiles, dificultando el modelado.
- Los errores de las series base se propagan y acumulan en los niveles agregados.

Métodos Principales de Forecasting

Top-Down

Idea:

- Lo contrario a Bottom-Up

Primero

Primero se pronostica el nivel superior:
Total turistas: 1000

Luego

Después se distribuye hacia abajo usando proporciones históricas.

Históricamente

Hotel: 60%
Bus: 30%
Otros: 10%



Entonces

Hotel = 600
Bus = 300
Otros = 100

Ventajas

- Funciona bien cuando las series base son muy ruidosas o presentan pocos datos.
- Solo requiere pronosticar la serie agregada, reduciendo la complejidad del modelado.
- Produce pronósticos más estables y precisos en los niveles superiores.

Desventajas

- Pierde la información específica de cada serie (promociones, efectos locales, estacionalidad).
- La desagregación mediante proporciones puede introducir sesgo y no capturar cambios en la composición de las series.
- La precisión en los niveles inferiores suele ser limitada al depender de reglas de distribución.

Métodos de Reconciliación

Idea:

- Cada serie genera su propio forecast.
- Después se ajustan para que sean coherentes.

Forecast Independiente

Total = 1100

Hotel = 700

Bus = 300

Otros = 200

Suma de componentes = 1200

Algoritmo de Reconciliación

Total = 1100

Hotel = 650

Bus = 280

Otros = 170

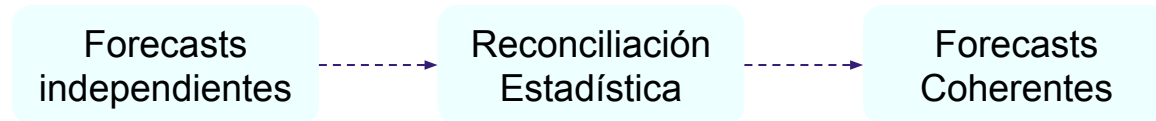
Suma de componentes = 1100

Métodos:

Bottom-Up OLS (Ordinary Least Squares) WLS (Weighted Least Squares) MinT (Minimum Trace)

Reconciliación Estadística

- Proceso que ajusta pronósticos independientes para que sean coherentes con la estructura jerárquica, modificándolos lo menos posible.



- El objetivo es encontrar un nuevo conjunto de pronósticos que:
 - Cumpla las restricciones jerárquicas.
 - Preserve al máximo la información del modelo original.
 - Minimice el error global del sistema.
- La reconciliación no genera nuevos pronósticos; ajusta los existentes para que respeten la jerarquía utilizando criterios estadísticos.

Reconciliación Estadística

Métodos

OLS (Ordinary Least Squares)

Ajusta los pronósticos minimizando el error cuadrático.

Simple y rápido.

Asume igual varianza para todas las series.

WLS (Weighted Least Squares)

Pondera cada serie según su varianza.

Da mayor peso a las series más confiables.

No considera correlaciones entre series

MinT (Minimum Trace)

Utiliza la matriz de covarianzas de los errores para minimizar la varianza total del forecast reconciliado.

Mayor precisión y estado del arte clásico.

Requiere estimar la matriz de covarianzas.

Reconciliación Estadística

MinT

MinT parte de pronósticos base obtenidos para todos los niveles de la jerarquía y luego los reconcilia para hacerlos coherentes.

La idea:

- Cada serie produce su propio forecast.
- Estos forecasts probablemente son inconsistentes.
- MinT los ajusta minimizando el error esperado.

Modelo Total

Total = 1000

Hotel = 700

Bus = 400

Suma de componentes = 1100

MinT

Recibe los pronósticos

$$\hat{y} = \begin{bmatrix} 1000 \\ 700 \\ 400 \end{bmatrix}$$

y busca nuevos valores:

$$\tilde{y}$$

que cumplan:

$$\tilde{y} = S\tilde{b}$$

Tenemos:

$$y = Sb$$

Los pronósticos originales son: $\hat{y}$

MinT busca: $\tilde{y} = SG\hat{y}$

Donde G es una matriz de reconciliación.

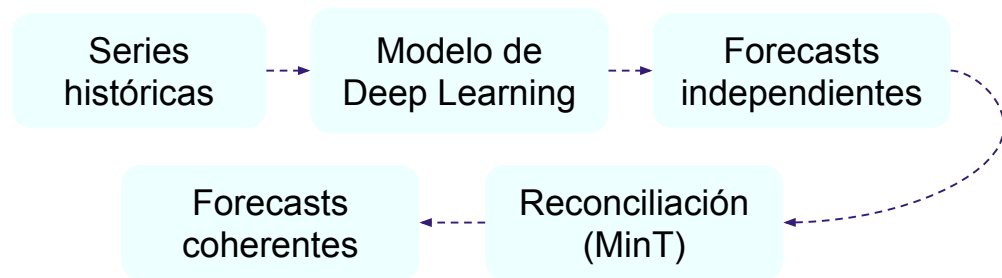
La solución minimiza:

$$Trace(Var(\tilde{y} - y))$$

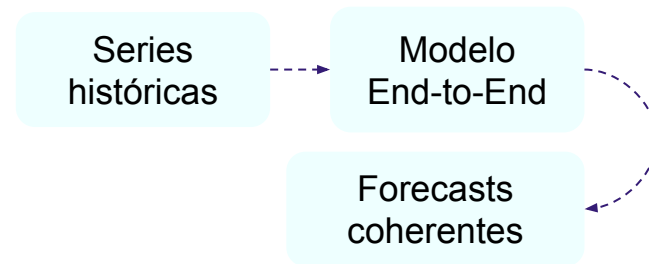
Evolución del Forecasting Jerárquico

- Los primeros enfoques se basaban en reglas de agregación (Bottom-Up, Top-Down y Middle-Out).
- Posteriormente surgieron los métodos de reconciliación estadística (OLS, WLS y MinT), que ajustan pronósticos independientes para garantizar coherencia.
- Actualmente, la investigación busca integrar predicción y reconciliación en un único modelo entrenado de extremo a extremo (end-to-end).

Enfoque clásico



Enfoque End-to-End



La reconciliación deja de ser una etapa separada.

Forecasting Jerárquico

[LINK](#)

